

TASK 1: COMPUTER VISION USING CNN MODELS

Name: Sivaranjani.S

Registration Number: INBT019473

Internship Program: iNeuBytes(AI)

1. Introduction

Image classification is one of the most important applications of Deep Learning and Computer Vision. It involves assigning a class label to an input image using a Convolutional Neural Network (CNN). CNNs automatically learn features such as edges, textures, and object shapes directly from images without manual feature extraction.

In this task, two CNN models were implemented and compared:

- Traditional CNN
- Customized CNN

The objective was to analyze the performance of both models and identify which architecture provides better classification accuracy.

2. Objective

The objectives of this project are:

- To understand Convolutional Neural Networks.
- To build a Traditional CNN model.
- To design a Customized CNN architecture.
- To train both models on the image dataset.
- To compare the performance using various evaluation metrics.
- To analyze why the customized CNN performs better than the traditional CNN.

3. Dataset

The image dataset consists of multiple classes of images used for classification.

The dataset was divided into:

- Training Dataset
- Validation Dataset
- Testing Dataset

Before training, all images were resized to 32×32 pixels and normalized to improve model performance.

4. Software Requirements

- Python
- Google Colab / VS Code
- TensorFlow
- Keras
- NumPy
- Matplotlib
- Scikit-learn

5. Traditional CNN Architecture

The traditional CNN consists of:

- Input Layer
- Convolution Layer
- ReLU Activation
- Max Pooling Layer
- Flatten Layer
- Dense Layer
- Output Layer using Softmax

This model serves as the baseline architecture.

6. Customized CNN Architecture

The customized CNN was developed by modifying the traditional CNN.

Additional improvements include:

- More Convolution Layers
- Batch Normalization
- Dropout Layers
- Increased Filters
- Better Feature Extraction
- Improved Generalization

These modifications reduce overfitting and improve classification accuracy.

7. Methodology

The project follows these steps:

- **Step 1**
- Import required Python libraries.
- **Step 2**
- Load the image dataset.
- **Step 3**
- Preprocess the images.
- **Step 4**
- Split the dataset into training, validation, and testing sets.
- **Step 5**
- Build the Traditional CNN model.
- **Step 6**
- Train the Traditional CNN.
- **Step 7**
- Build the Customized CNN.
- **Step 8**
- Train the Customized CNN.
- **Step 9**
- Evaluate both models.
- **Step 10**
- Compare performance using evaluation metrics.

8. Performance Evaluation

The models were evaluated using:

- Accuracy

- Precision
- Recall
- F1-Score
- Confusion Matrix

Training and validation accuracy/loss graphs were also generated.

9. Results

Both CNN models successfully classified the images.

The Customized CNN produced better classification results compared to the Traditional CNN due to its improved architecture.

10. Comparison

Parameter	Traditional CNN	Customized CNN
Accuracy	Higher than random baseline	Higher than Traditional CNN
Precision	Good	Better
Recall	Good	Better
F1-Score	Good	Better
Training Time	Lower	Slightly Higher
Performance	Good	Excellent

11. Why Customized CNN Performed Better

The Customized CNN achieved better performance because:

- More convolution layers extracted richer image features.
- Batch Normalization stabilized training.
- Dropout reduced overfitting.
- Additional filters captured more complex patterns.
- Better feature learning improved prediction accuracy.

Overall, the customized architecture generalized better to unseen images.

12.Conclusion

In this task, both Traditional CNN and Customized CNN models were successfully implemented for image classification. After training and evaluation, the Customized CNN outperformed the Traditional CNN by achieving better accuracy, precision, recall, and F1-score. The project demonstrates the effectiveness of deep learning techniques in solving image classification problems and highlights how architectural improvements can significantly enhance model performance.